A Comparative Study of Audio Feature Extraction and Machine Learning Methods for Bird Species Classification*

Kryspin Dziarek[1], Szymon Pająk[1] and Marcin Adamski[1]

[1]*Faculty of Applied Mathematics, Silesian University of Technology, Kaszubska 23, 44100 Gliwice, POLAND*

Abstract

do uzupelnienia do uzupelnienia

wstep : 1 akapit o tym co zrobiono, jakie wyniki, minimum 180 słów

Keywords

Sound analysis, Birdsong, MFCC (Mel-Frequency Cepstral Coefficients), Random forest, Neural network

1. Introduction

Audio analysis is a process of examining and measuring sound signals to gain information, it has been a subject of research for many decades[1], and now, in the age of artificial intelligence, it has many more applications. It allows the interaction between humans and machines through sound, making advanced audio-based systems possible. Such systems include voice assistants, audio user interfaces (AUI) and voice verification. Machine learning has become an essential tool in audio signal processing, enabling the development of systems capable of analyzing and classifying sounds. Such systems are widely used in applications including speech recognition[2], music[3] and animal identification[4], often outperforming human perception of acoustic patterns[5].

There are many techniques for feature extraction from sound, one of the most commonly used being Mel-Frequency Cepstral Coefficients (MFCC). This method converts the raw audio signal into a set of coefficients representing the spectral characteristics of sound[6]. Previous studies have established standard configurations for this algorithm, such as the number of Mel filters and cepstral coefficients. However, optimal values depend on specific dataset and application, such as detection of breathing phase[7] or respiratory diseases[8].

In addition to sound recognition, various machine learning algorithms have been applied to audio classification tasks, each providing different levels of performance. Simpler algorithms, such as k-nearest neighbors and random forests, are faster, but often less accurate solutions, while the complex ones, like neural networks, provide higher accuracy at the cost of increased computational complexity.

SSI, project 2025/2026

kd315963@student.polsl.pl (K. Dziarek); sp315960@student.polsl.pl (S. Pająk); ma315929@student.polsl.pl (M. Adamski)

This paper presents bird species recognition with sound analysis (using MFCC) and compares different approaches for this system. The sound analysis algorithm consists of many stages, two of them are especially important, because the values they provide depend on chosen data. The number of filters used to describe the frames and number of compressed coefficients strongly affects the performance of the result. Too many features may introduce noise and redundancy, while too few may lead to information loss. The experiments conducted on a common dataset aim to identify the most effective approach for extracting the MFCC features and developing simple classification models with the best performance.

2. Methodology

This section outlines the approach used to compare different configurations of MFCC in audio feature extraction and the development of machine learning models for identifying bird species.

2.1. Audio Feature Extraction

Audio feature extraction enables the transformation of birdsong recordings into numerical representations, which are crucial for training machine learning models. A custom library called *SoundFeatures* was developed to implement the full extraction pipeline. This section describes its structure and role within the experimental setup.
Audio files were converted from MP3 to FLAC format to provide a consistent and lossless compression of the signal. Although this conversion does not restore the data lost during MP3 compression, it eliminates decoding artifacts and ensures stable input for further signal processing [9].

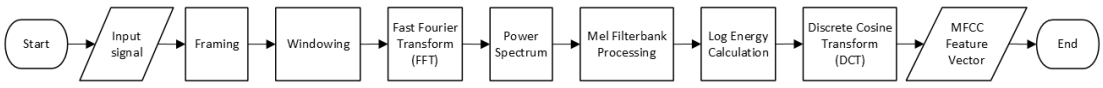


Figure 1: Audio feature extraction process flowchart

The audio feature extraction process consists of the following steps (??):

1. **Framing** - Segmenting the signal into short, overlapping frames.
2. **Windowing** - Applying a window function to each frame to minimize spectral leakage at the boundaries. The experiments consisted of Hamming window function.

$$w_{\text{Hamming}}[k] = 0.54 - 0.46 \cos\left(\frac{2\pi k}{N-1}\right)$$

3. **Fast Fourier Transform (FFT)** - Mapping a time-domain signal into its frequency-domain representation.

$$A_k = \sum_{p=0}^{n-1} a_p e^{-\frac{i2\pi kp}{n}}$$

where:

- i - imaginary unit
- k - frequency bin index
- n - sample count in frame a (frame length)
- p - sample index in the time domain
- a_p - sample at index p in frame a

4. **Power Spectrum** - Computing the energy distribution representation.

$$\tilde{P}_k = \frac{|A_k|^2}{N}$$

where:

- A_k - output of the DFT at frequency bin k
- k - frequency bin index
- N - frame length
- $\tilde{P}_k$ - Normalized power spectrum

5. **Mel Filterbank Processing** - Constructing a set of triangular filters using Mel scale. This process consists of transferring a vector containing equally distributed Mel-frequency values in the range from $Mel(f_{min})$ to $Mel(f_{max})$ back to Herzian scale to develop the filterbank.

$$Mel(f) = 2595 \log_{10}\left(1 + \frac{f}{700}\right) \Rightarrow f = 700 \cdot 10^{\frac{Mel(f)}{2595}} - 1$$

where:

- f - frequency in Hz
- $Mel(f)$ - frequency in the Mel scale

$$H_m(k) = \begin{cases} 0, & k < f_{m-1} \vee k > f_{m+1} \\ \dfrac{k - f_{m-1}}{f_m - f_{m-1}}, & f_{m-1} \leq k \leq f_m \\ \dfrac{f_{m+1} - k}{f_{m+1} - f_m}, & f_m \leq k \leq f_{m+1} \end{cases}$$

where:

- m - filter number
- k - index of sample frequency
- f - vector of filters in Herzian scale
- H - Mel filterbank

6. **Log Energy Calculation** - Mapping the linear frequency spectrum to Mel scale using Mel filterbank.

$$S_m^{log} = \log_{10}\left(\sum_k P_k \cdot H_m(k)\right)$$

7. **Discrete Cosine Transform (DCT)** - Decorrelating the log energies to get Mel-Frequency Cepstral Coefficients

$$c_m = \sum_{p=0}^{k-1} S_m^{log} \cdot \cos\left(\frac{\pi m(p+0.5)}{k}\right)$$

where:

- m - Mel coefficient index
- k - Chosen count of Mel coefficients
- c - Mel coefficient vector

2.2. Machine Learning Algorithms

2.2.1. Random Forest

The Random Forest algorithm was chosen for classification purposes, a well-known ensemble learning method originally introduced by Leo Breiman [10]. Essentially, instead of relying on just one model, it constructs a whole bunch of individual decision trees during the training phase and aggregates their predictions to get the final output. Since we are dealing with classification, the model determines the final predicted class by running a majority vote across all the trees in the forest:

$$\hat{y} = \arg\max_c \sum_{t=1}^{T} \mathbf{1}\left[h_t(\mathbf{x}) = c\right] \tag{1}$$

In this equation, T stands for the total number of trees, $h_t(x)$ is the prediction made by the t-th tree for our input x, and c represents the possible classes we are trying to predict.

To make the ensemble robust and prevent the trees from being too similar, each tree is trained on what is called a "bootstrap sample"—basically, a random subset of our training data selected with replacement. On top of that, when splitting a node, the algorithm doesn't look at all available features at once. Instead, it only considers a random subset of m features out of the total p features. This clever trick decorrelates the individual trees, which drastically cuts down the overall variance of the model [10].

To pick the best possible split at any given node, the algorithm evaluates how "pure" the resulting splits will be by minimizing the Gini impurity:

$$G(S) = 1 - \sum_{c=1}^{C} p_c^2 \tag{2}$$

Here, S is the set of samples currently at the node, C represents the number of classes, and p_c is the proportion of samples belonging to class c. The actual split into left (S_L) and right (S_R) subsets is chosen to maximize the information gain (ΔG):

$$\Delta G = G(S) - \frac{|S_L|}{|S|}G(S_L) - \frac{|S_R|}{|S|}G(S_R) \tag{3}$$

One of the biggest practical advantages of using Random Forests is something called the out-of-bag (OOB) error estimate. Because of how the bootstrap sampling works, roughly 36.8% of the training data is left out of any single tree's training process on average. We can actually use these left-out samples as an internal validation set. This has a couple of advantages an unbiased estimate of how well our model generalizes without needing to sacrifice data for a separate validation split [10]:

$$\text{OOB Error} = \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}\left[\hat{y}_i^{\text{OOB}} \neq y_i\right] \tag{4}$$

In this formula, $\hat{y}_i^{OOB}$ is the majority vote prediction for sample i using only the specific trees that did not see this sample during their training, and N is the total number of training samples.

When it comes to classifying bird species from MFCC data, Random Forest gives us a couple of great advantages: its built-in feature importance scores make it easy to identify which spectral coefficients actually matter, and its ensemble structure keeps it from getting easily thrown off by noisy or irrelevant audio features [11, 12].

2.2.2. Neural Network

The second classification model used in this study was a neural network. The concept of artificial neural networks dates back to the work of McCulloch and Pitts, who proposed an early mathematical model of a neuron in 1943 [13]. The model used in this work belongs to the class of feedforward neural networks, also known as multilayer perceptrons. Such models are composed of interconnected computational units called neurons, organized into an input layer, one or more hidden layers, and an output layer [14]. The input layer receives the MFCC feature vector, hidden layers transform it into internal representations, and the output layer produces the final class prediction.
Each layer applies a linear transformation followed by a non-linear activation function:

$$z = XW + b \tag{5}$$

$$a = f(z) \tag{6}$$

where X is the input matrix, W is the weight matrix, b is the bias vector, f is the activation function, z is the linear output of the layer, and a is the activated output. The weights determine the strength of connections between neurons, while the bias terms allow the activation to be shifted independently of the input values.
The activation function introduces non-linearity into the network. Without a non-linear activation function, a sequence of dense layers would be equivalent to a single linear transformation,

limiting the model's ability to learn complex relationships among MFCC features. In the hidden layers, ReLU or Leaky ReLU activation functions were used. ReLU is defined as:

$$f_{\text{ReLU}}(z) = \max(0, z) \tag{7}$$

This function passes positive values unchanged and sets negative values to zero. Leaky ReLU is a modified version of ReLU:

$$f_{\text{LeakyReLU}}(z) = \begin{cases} z, & z > 0 \\ \alpha z, & z \leq 0 \end{cases} \tag{8}$$

where α is a small positive coefficient, equal to 0.01 in the implemented model. Unlike standard ReLU, Leaky ReLU keeps a small slope for negative values, which helps preserve gradient flow during training [14].

For a multi-class classification problem, the output layer was configured to contain one neuron for each bird species. The softmax function was used to transform the raw output values into probabilities:

$$\hat{y}_c = \frac{e^{z_c}}{\sum_{j=1}^{C} e^{z_j}} \tag{9}$$

where C is the number of classes, z_c is the output value for class c, and $\hat{y}_c$ is the predicted probability of this class. The predicted class is selected as the class with the highest probability:

$$\hat{y} = \arg\max_c \hat{y}_c \tag{10}$$

The model was trained by minimizing the categorical cross-entropy loss function:

$$L = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c} \log(\hat{y}_{i,c}) \tag{11}$$

where N is the number of samples, C is the number of classes, $y_{i,c}$ is the true label in one-hot representation, and $\hat{y}_{i,c}$ is the predicted probability for class c. This loss function is suitable for multi-class classification because it penalizes confident incorrect predictions.

During training, the error gradient was propagated backwards through the network using backpropagation. This procedure computes how much each parameter contributes to the final error and updates the weights in order to reduce the loss. The implemented network supports both stochastic gradient descent and the Adam optimizer. In the experiments, Adam was used to update the model parameters, because it adapts the learning rate separately for each parameter using estimates of the first and second moments of the gradients [15].

The training process was performed using mini-batches. For each batch, the network computed predictions, evaluated the cross-entropy loss, propagated the gradient backwards and updated the model parameters. During training, both loss and accuracy were monitored for the training and validation sets.

To reduce the risk of overfitting, dropout was applied in the hidden layer. Dropout is a regularization technique in which randomly selected neuron activations are temporarily set to zero

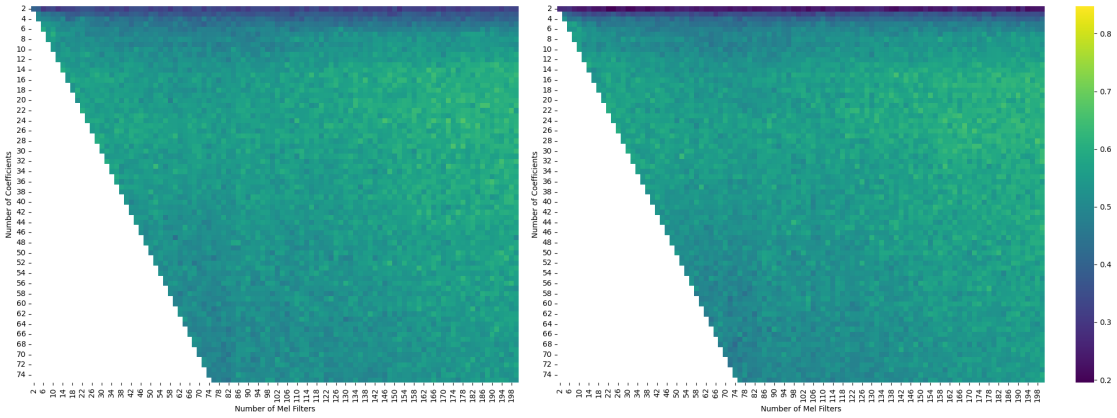
during training. As a result, the network cannot rely too strongly on individual neurons and is forced to learn more robust feature representations [16]. In the implemented model, dropout was applied only during training and was not used in the output softmax layer. The remaining activations were scaled to preserve the expected signal magnitude.

3. Experiments

brakuje opisu zbioru danych, podzialu danych, jak również informacji o wykorzystanym sprzecie do obliczeń

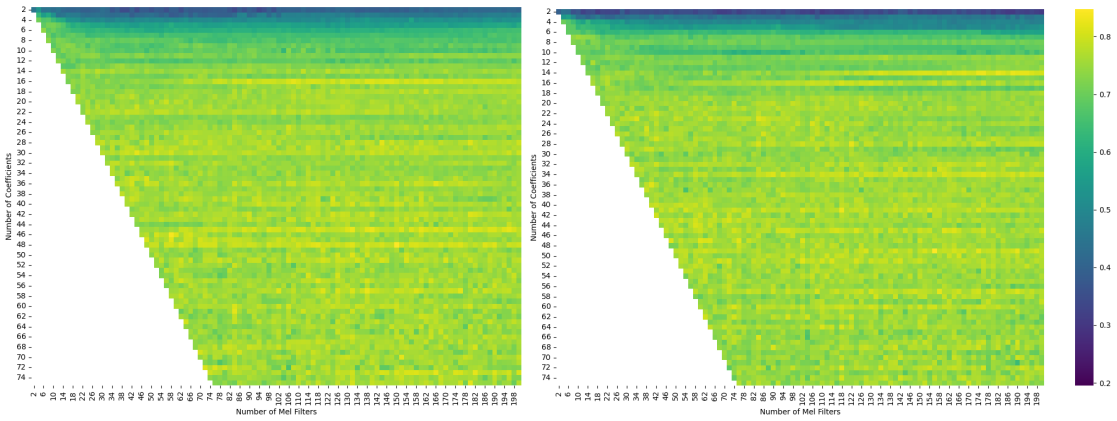
3.1. Audio Feature Extraction

To analyse the MFCC efficiency based on the number of Mel filters and extracted features the experiments were conducted on dataset which was reduced to 1148 records. Both the Random Forest and the Neural Network were applied to check the F1 score for every configuration.



(a) Dataset with every coefficient (b) Dataset without first coefficient

Figure 2: Random Forest F1 Score heatmaps for datasets with and without first coefficient



(a) Dataset with every coefficient (b) Dataset without first coefficient

Figure 3: Neural Network F1 Score heatmaps for datasets with and without first coefficient

While standard speech recognition systems typically employ 13 Mel coefficients and between 26 and 40 Mel filters[17, 18, 19], our results demonstrate that bird species identification requires

Figure 4: Best results

Algorithm	N Coefficients	N Filters	First Coefficient Ignored	F1 Score
Random Forest	24	178	No	0.6669
Random Forest	28	198	Yes	0.6618
Neural Network	48	150	No	0.8472
Neural Network	49	156	Yes	0.8454

a higher spectral resolution. The optimal configuration for the Neural Network was observed in the range of 45-49 Mel coefficients with at least 58 Mel filters (3), whereas the Random Forest achieved its best performance with 21-25 Mel coefficients and at least 146 Mel filters (2). These results emphasize the fact that birdsong contains faster frequency modulations and more complex harmonic structures compared to human speech, requiring a denser filterbank to effectively capture significant parts of the sound, which is essential for accurate species discrimination. Both machine learning algorithms achieved higher results analysing every coefficient, including the first one, which represents the volume. The first coefficient often reaches massive values, dominating other coefficients and making machine learning performance dependent on the distance of microphone. However, experiments have shown, that including this coefficient in examined dataset actually improves the efficiency (4).
The difference in the optimal spectral resolutions between the two models highlights the higher sensitivity of the Random Forest to excessive feature counts, which can degrade its performance. In contrast, the Neural Network effectively manages high-dimensional inputs, achieving better performance by capturing more complex patterns within the given data.

3.2. Random Forest

Figure 5: Comparison of Random Forest variants with 100 trees

Variant	Accuracy	Precision	Recall	Specificity	F1 Score	Error
RF(gini, sqrt)	0.2766	0.2749	0.2767	0.9919	0.2614	0.7234
RF(gini, log2)	0.2714	0.2753	0.2722	0.9918	0.2566	0.7286
RF(gini, 2)	0.2429	0.2400	0.2426	0.9915	0.2243	0.7571
RF(entropy, sqrt)	0.2435	0.2393	0.2418	0.9915	0.2266	0.7565
RF(entropy, log2)	0.2481	0.2469	0.2432	0.9916	0.2306	0.7519
RF(entropy, 2)	0.2254	0.2250	0.2233	0.9913	0.2101	0.7746

To select the best model, the algorithms were trained on various parameters. The most important were the criteria (gini, entropy) and max_features (sqrt, log2, constant). Due to the high power costs and time-consuming nature of Random Forest implementation, the number of trees was set to 100, and the constant for max_features was set to 2 (5).

Once the best model was found, a search began for the largest number of trees in the forest to improve the algorithm's accuracy. The number of trees was checked successively for 100, 200, 300, 400, and 500 trees (6). The two models with the highest accuracies were then taken, their tree counts were added, their tree counts were divided by two, and the integer part was taken.

Figure 6: Comparison of the number of trees in a forest

Number of trees	Accuracy	Precision	Recall	Specificity	F1 Score	Error
100	0.2766	0.2749	0.2767	0.9919	0.2614	0.7234
200	0.2927	0.2960	0.2907	0.9921	0.2746	0.7073
300	0.3070	0.3120	0.3100	0.9922	0.2894	0.6930
400	0.3180	0.3267	0.3198	0.9923	0.3011	0.6820
500	0.3161	0.3227	0.3169	0.9923	0.2672	0.6839
450	0.3161	0.3224	0.3180	0.9923	0.2986	0.6839
425	0.3187	0.3294	0.3205	0.9923	0.3018	0.6813
437	0.3187	0.3272	0.3203	0.9923	0.3013	0.6813
443	0.3180	0.3236	0.3195	0.9923	0.2998	0.6820
440	0.3199	0.3260	0.3218	0.9924	0.3022	0.6801
441	0.3180	0.3236	0.3198	0.9923	0.2999	0.6820

The process was repeated until the appropriate number of trees for the highest accuracy was found. If the same result was found for both tree counts, the number of trees that was closer to the number of trees in the second component with the second highest number of trees was used.

3.3. Neural Network

proszę dodać ajką analizę innych metryk typu accuracy, error, analizę sieci neuronowej - ile warstw, dlaczego, co jesli sie zmniejszy ta ilosc itd.

4. Conclusion

do uzupelnienia

References

[1] J. D. Hollabaugh, Methods for phonemic recognition in speech processing (1963).

[2] M. Zhu, C. Wang, A systematic review of research on ai in language education: Current status and future implications (2025).

[3] S. Serrano, M. A. B. Sahbudin, C. Chaouch, M. Scarpa, A new fingerprint definition for effective song recognition, Pattern Recognition Letters 160 (2022) 135–141.

[4] C. M. Wood, A. Barceinas Cruz, S. Kahl, Pairing a user-friendly machine-learning animal sound detector with passive acoustic surveys for occupancy modeling of an endangered primate, American Journal of Primatology 85 (2023) 269–307.

[5] C. Spille, B. Kollmeier, B. T. Meyer, Comparing human and automatic speech recognition in simple and complex acoustic scenes, Computer Speech & Language 52 (2018) 123–140.

[6] Z. K. Abdul, A. K. Al-Talabani, Mel frequency cepstral coefficient and its applications: A review, Ieee Access 10 (2022) 122136–122158.

[7] A. K. Zhantleuova, Y. K. Makashev, N. T. Duzbayev, Optimizing mfcc parameters for breathing phase detection, Sensors 25 (2025) 5002.
[8] Y. Yan, S. O. Simons, L. van Bemmel, L. G. Reinders, F. M. Franssen, V. Urovi, Optimizing mfcc parameters for the automatic detection of respiratory diseases, Applied Acoustics 228 (2025) 110299.
[9] G. Shi, Data compression for audio-based smart beekeeping (2024) 50–52.
[10] L. Breiman, Random forests, Machine Learning 45 (2001) 5–32. doi:10.1023/A:1010933404324.
[11] Y. Huang, et al., Recognition of bird species with birdsong records using machine learning methods, PLoS ONE 19 (2024). doi:10.1371/journal.pone.0297988.
[12] S. Ahmed, et al., Bird species identification from audio data (2023). URL: https://www.researchgate.net/publication/373612045.
[13] W. S. McCulloch, W. Pitts, A logical calculus of the ideas immanent in nervous activity, The Bulletin of Mathematical Biophysics 5 (1943) 115–133.
[14] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016. Available at: https://www.deeplearningbook.org/.
[15] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, in: International Conference on Learning Representations, 2015.
[16] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, R. Salakhutdinov, Dropout: A simple way to prevent neural networks from overfitting, Journal of Machine Learning Research 15 (2014) 1929–1958.
[17] S. C. Joshi, A. Cheeran, Matlab based feature extraction using mel frequency cepstrum coefficients for automatic speech recognition, International Journal of Science, Engineering and Technology Research (IJSETR) 3 (2014) 1820.
[18] A. Mahmood, U. Köse, Speech recognition based on convolutional neural networks and mfcc algorithm, Advances in Artificial Intelligence Research 1 (2021) 6–12.
[19] S. M. Al Sasongko, S. Tsaury, S. Ariessaputra, S. Ch, Mel frequency cepstral coefficients (mfcc) method and multiple adaline neural network model for speaker identification, JOIV: International Journal on Informatics Visualization 7 (2023) 2306–2312.